

DATA ANALYST INTERNSHIP

TASK 6: Sales Trend Analysis

Objective:

Analyze **monthly revenue** and **order volume** using SQL aggregation functions, grouping, and time-based analysis to identify sales trends and top-performing periods.

Dataset Used:

online_sales.csv

(Contains order_id, order_date, amount, product_id, product_name, cust_id, and city columns)

Tools & Technologies:

- MySQL Workbench
 - SQL (DML & Aggregate Functions)
 - CSV Dataset Import
 - Data Aggregation & Grouping Techniques
-

Deliverables:

- SQL Script containing all queries (task6_sales_trend_analysis.sql).
 - PDF Report summarizing findings with Quer Screenshots of key outputs.
-

Key Functions Used:

STR_TO_DATE(), MONTH(), YEAR(), SUM(), COUNT(DISTINCT), GROUP BY, ORDER BY, LIMIT

Submitted By:

Name: Sourav Banerjee

Role: Data Analyst

SQL_QUERIES

-- 1. Display the first 10 rows

```
SELECT * FROM online_sales LIMIT 10;
```

-- 2. Convert order_date (text) to date format

```
SELECT order_id, order_date,  
       STR_TO_DATE(order_date, '%Y-%m-%d') AS converted_date  
FROM online_sales  
LIMIT 10;
```

-- 3. Extract month and year from order_date

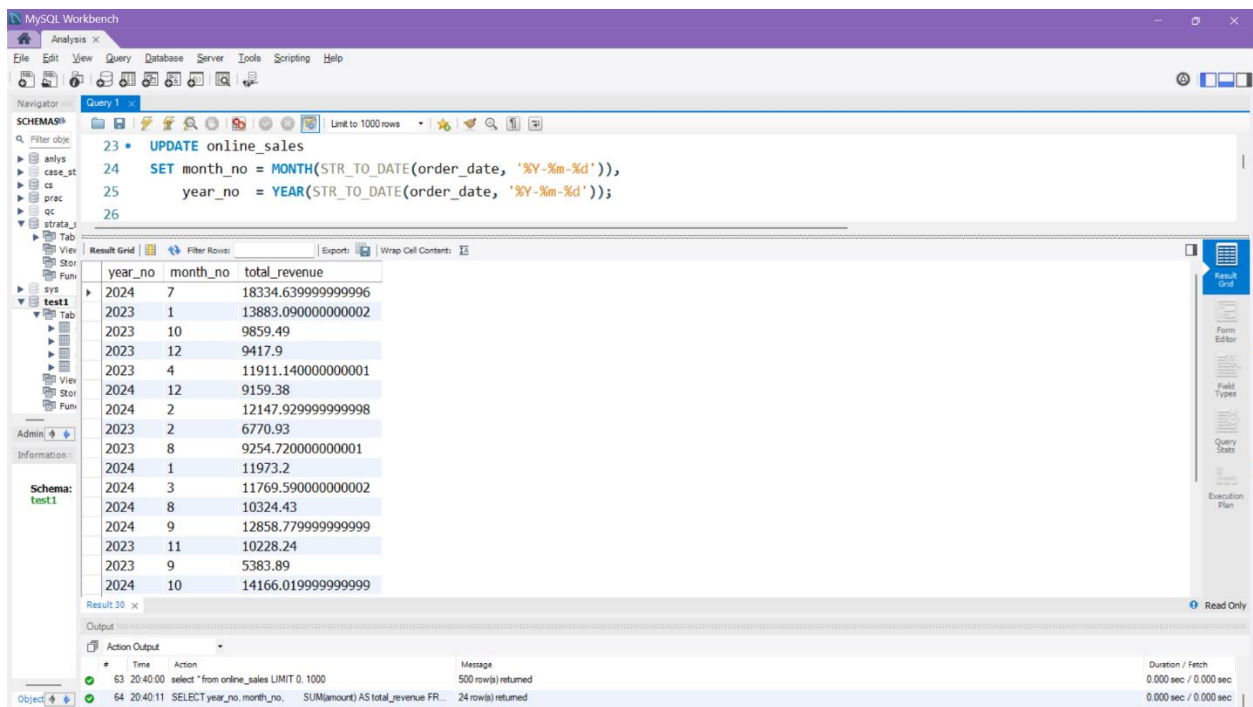
```
SELECT order_id,  
       MONTH(STR_TO_DATE(order_date, '%Y-%m-%d')) AS month_no,  
       YEAR(STR_TO_DATE(order_date, '%Y-%m-%d')) AS year_no  
FROM online_sales  
LIMIT 10;
```

-- 4. Add month_no and year_no columns

```
ALTER TABLE online_sales  
ADD COLUMN month_no INT,  
ADD COLUMN year_no INT;  
  
UPDATE online_sales  
SET month_no = MONTH(STR_TO_DATE(order_date, '%Y-%m-%d')),  
    year_no = YEAR(STR_TO_DATE(order_date, '%Y-%m-%d'));
```

-- 5. Total revenue per month

```
SELECT year_no, month_no,  
  
       SUM(amount) AS total_revenue  
  
FROM online_sales  
  
GROUP BY year_no, month_no;
```



The screenshot shows the MySQL Workbench interface. The query editor contains an SQL query to update the 'online_sales' table, setting 'month_no' and 'year_no' based on the 'order_date'. The result grid displays the output of the query, showing columns 'year_no', 'month_no', and 'total_revenue' with 24 rows of data.

year_no	month_no	total_revenue
2024	7	18334.639999999996
2023	1	13883.090000000002
2023	10	9859.49
2023	12	9417.9
2023	4	11911.140000000001
2024	12	9159.38
2024	2	12147.929999999998
2023	2	6770.93
2023	8	9254.720000000001
2024	1	11973.2
2024	3	11769.590000000002
2024	8	10324.43
2024	9	12858.779999999999
2023	11	10228.24
2023	9	5383.89
2024	10	14166.019999999999

-- 6. Total order volume per month

```
SELECT year_no, month_no,  
  
       COUNT(DISTINCT order_id) AS total_orders  
  
FROM online_sales  
  
GROUP BY year_no, month_no;
```

-- 7. Revenue + order volume together

```
SELECT year_no, month_no,
```

SUM(amount) AS total_revenue,

COUNT(DISTINCT order_id) AS total_orders

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
35 -- 6. Total order volume per month
36 * SELECT year_no, month_no,
37        COUNT(DISTINCT order_id) AS total_orders
38 FROM online sales
```

The Results Grid displays the following data:

year_no	month_no	total_revenue	total_orders
2023	1	13883.090000000002	25
2023	2	6770.93	18
2023	3	8268.949999999999	17
2023	4	11911.140000000001	23
2023	5	17479.090000000004	35
2023	6	13580.179999999998	23
2023	7	8811.170000000002	17
2023	8	9254.720000000001	15
2023	9	5383.89	13
2023	10	9859.49	22
2023	11	10228.24	21
2023	12	9417.9	17
2024	1	11973.2	24
2024	2	12147.929999999998	22
2024	3	11769.590000000002	21
2024	4	9696.369999999999	18

The bottom panel shows the Action Output log with the following entries:

#	Time	Action	Message	Duration / Fetch
64	20:40:11	SELECT year_no, month_no, SUM(amount) AS total_revenue FR...	24 row(s) returned	0.000 sec / 0.000 sec
65	20:40:49	SELECT year_no, month_no, SUM(amount) AS total_revenue, ...	24 row(s) returned	0.000 sec / 0.000 sec

FROM online_sales

GROUP BY year_no, month_no;

-- 8. Sort results chronologically

SELECT year_no, month_no,

SUM(amount) AS total_revenue,

COUNT(DISTINCT order_id) AS total_orders

FROM online_sales

GROUP BY year_no, month_no

ORDER BY year_no, month_no;

MySQL Workbench

Query 1

```

47
48 -- 8. Sort results chronologically
49 SELECT year_no, month_no,
50 SUM(amount) AS total_revenue,
51 COUNT(DISTINCT order_id) AS total_orders
52 FROM online_sales
53 GROUP BY year_no, month_no
54 ORDER BY year_no, month_no

```

year_no	month_no	total_revenue	total_orders
2023	1	13883.090000000002	25
2023	2	6770.93	18
2023	3	8268.949999999999	17
2023	4	11911.140000000001	23
2023	5	17479.090000000004	35
2023	6	13580.179999999998	23
2023	7	8811.170000000002	17
2023	8	9254.720000000001	15
2023	9	5383.89	13
2023	10	9859.49	22
2023	11	10228.24	21
2023	12	9417.9	17

Result 33

Output

#	Time	Action	Message	Duration / Fetch
66	20:42:03	SELECT year_no, month_no, SUM(amount) AS total_revenue, COUNT(DISTINCT order_id) AS total_orders FROM online_sales GROUP BY year_no, month_no ORDER BY year_no, month_no	... 24 row(s) returned	0.000 sec / 0.000 sec
67	20:42:07	SELECT year_no, month_no, SUM(amount) AS total_revenue, COUNT(DISTINCT order_id) AS total_orders FROM online_sales GROUP BY year_no, month_no ORDER BY year_no, month_no	... 24 row(s) returned	0.000 sec / 0.000 sec

-- 9. Only for 2023

```

SELECT year_no, month_no,
SUM(amount) AS total_revenue,
COUNT(DISTINCT order_id) AS total_orders
FROM online_sales
WHERE year_no = 2023
GROUP BY year_no, month_no
ORDER BY month_no;

```

-- 10. Only for 2024

```

SELECT year_no, month_no,
SUM(amount) AS total_revenue,
COUNT(DISTINCT order_id) AS total_orders
FROM online_sales

```

WHERE year_no = 2024

GROUP BY year_no, month_no

ORDER BY month_no;

-- 11. First 6 months of 2023

SELECT year_no, month_no,

SUM(amount) AS total_revenue,

COUNT(DISTINCT order_id) AS total_orders

FROM online_sales

WHERE year_no = 2023 AND month_no BETWEEN 1 AND 6

GROUP BY year_no, month_no

ORDER BY month_no;

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
75  Execute the selected portion of the script or everything, if there is no selection
76  COUNT(DISTINCT order_id) AS total_orders
77  FROM online_sales
78  WHERE year_no = 2023 AND month_no BETWEEN 1 AND 6
79  GROUP BY year_no, month_no
80  ORDER BY month_no;
81  -- 12. Top 3 months by revenue
```

The Results Grid displays the following data:

year_no	month_no	total_revenue	total_orders
2023	1	13883.090000000002	25
2023	2	6770.93	18
2023	3	8268.949999999999	17
2023	4	11911.140000000001	23
2023	5	17479.090000000004	35
2023	6	13580.179999999998	23

The bottom panel shows the Action Output with the following messages:

```
67 20:42:07 SELECT year_no, month_no, SUM(amount) AS total_revenue, ... 24 row(s) returned
68 20:43:02 SELECT year_no, month_no, SUM(amount) AS total_revenue, ... 6 row(s) returned
```

-- 12. Top 3 months by revenue

SELECT year_no, month_no,

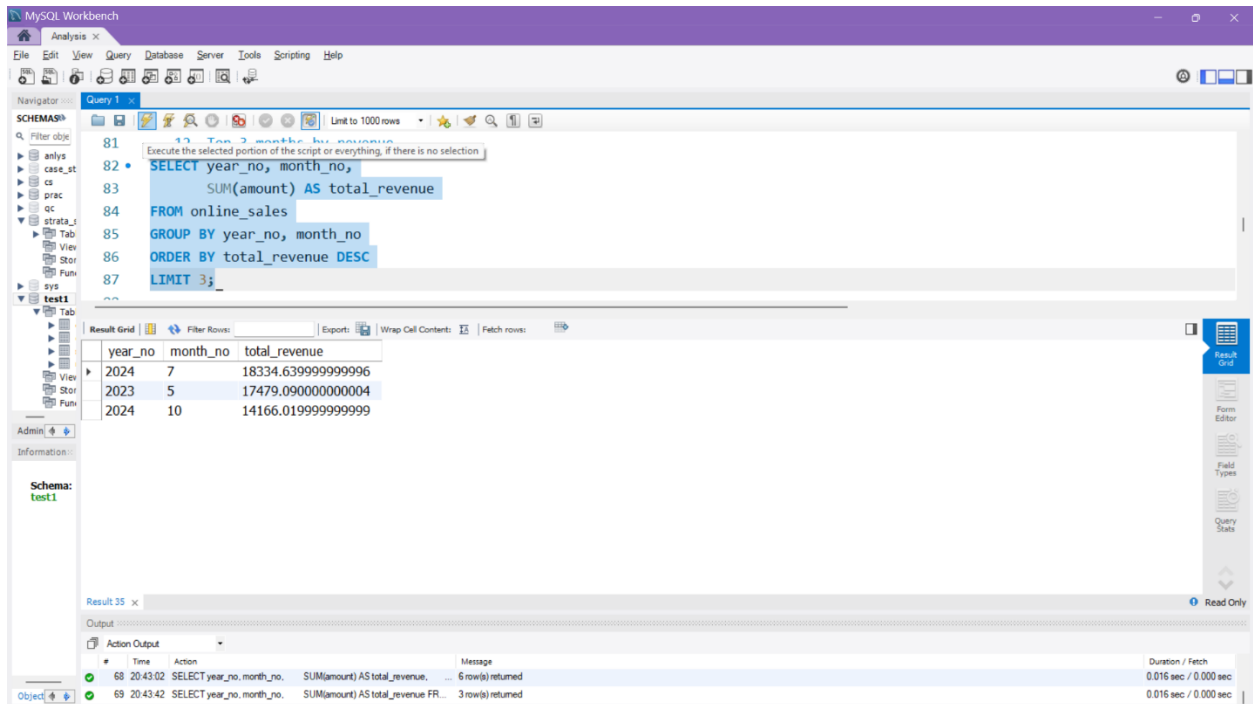
```
SUM(amount) AS total_revenue

FROM online_sales

GROUP BY year_no, month_no

ORDER BY total_revenue DESC

LIMIT 3;
```



```
-- 13. Top 3 months by order volume

SELECT year_no, month_no,
       COUNT(DISTINCT order_id) AS total_orders
FROM online_sales
GROUP BY year_no, month_no
ORDER BY total_orders DESC

LIMIT 3;
```

-- 14. Month with lowest revenue

```
SELECT year_no, month_no,  
       SUM(amount) AS total_revenue  
FROM online_sales  
GROUP BY year_no, month_no  
ORDER BY total_revenue ASC  
LIMIT 1;
```

-- 15. Month with lowest order volume

```
SELECT year_no, month_no,  
       COUNT(DISTINCT order_id) AS total_orders  
FROM online_sales  
GROUP BY year_no, month_no  
ORDER BY total_orders ASC  
LIMIT 1;
```

-- 16. Compare monthly revenue between 2023 and 2024

```
SELECT year_no, month_no,  
       SUM(amount) AS total_revenue  
FROM online_sales  
WHERE year_no IN (2023, 2024)  
GROUP BY year_no, month_no  
ORDER BY year_no, month_no;
```


MySQL Workbench

Query 1

```

112 16. Compute monthly revenue between 2023 and 2024
113 • SELECT year_no, month_no,
114       SUM(amount) AS total_revenue
115 FROM online_sales
116 WHERE year_no IN (2023, 2024)
117 GROUP BY year_no, month_no
118 ORDER BY year_no, month_no;

```

Result Grid

year_no	month_no	total_revenue
2023	1	13883.090000000002
2023	2	6770.93
2023	3	8268.949999999999
2023	4	11911.140000000001
2023	5	17479.090000000004
2023	6	13580.179999999998
2023	7	8811.170000000002
2023	8	9254.720000000001
2023	9	5383.89
2023	10	9859.49
2023	11	10228.24
2023	12	9417.9

Output

#	Time	Action	Message	Duration / Fetch
70	20:44:34	SELECT year_no, month_no, SUM(amount) AS total_revenue FR...	24 row(s) returned	0.000 sec / 0.000 sec
71	20:44:36	SELECT year_no, month_no, SUM(amount) AS total_revenue FR...	24 row(s) returned	0.015 sec / 0.000 sec

-- 17. Total revenue per city

```

SELECT city, SUM(amount) AS total_revenue
FROM online_sales
GROUP BY city
ORDER BY total_revenue DESC;

```

-- 18. Revenue per product per month

```

SELECT year_no, month_no, product_name,
       SUM(amount) AS total_revenue
FROM online_sales
GROUP BY year_no, month_no, product_name
ORDER BY year_no, month_no;

```

-- 19. Revenue per customer per month

```

SELECT year_no, month_no, cust_id,

       SUM(amount) AS total_revenue

FROM online_sales

GROUP BY year_no, month_no, cust_id

ORDER BY year_no, month_no;

```

-- 20. Create a summary view

```

CREATE VIEW v_sales_summary AS

SELECT year_no, month_no,

       COUNT(DISTINCT order_id) AS total_orders,

       SUM(amount) AS total_revenue

FROM online_sales

GROUP BY year_no, month_no

ORDER BY year_no, month_no;

```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```

135      SUM(amount) AS total_revenue
136  FROM online_sales
137  GROUP BY year_no, month_no, cust_id
138  ORDER BY year_no, month_no;
139
140  -- 20. Create a summary view
141  CREATE VIEW v_sales_summary AS

```

The Result Grid shows the data for the view `v_sales_summary`:

year_no	month_no	total_orders	total_revenue
2023	1	25	13883.0900000000002
2023	2	18	6770.93
2023	3	17	8268.9499999999999
2023	4	23	11911.1400000000001
2023	5	35	17479.0900000000004
2023	6	23	13580.1799999999998
2023	7	17	8811.1700000000002
2023	8	15	9254.7200000000001
2023	9	13	5383.89
2023	10	22	9859.49
2023	11	21	10228.24
2023	12	17	9417.9
2024	1	24	11973.2

The Output pane shows the execution results:

```

72 20:45:03 CREATE VIEW v_sales_summary AS SELECT year_no, m... 0 row(s) affected 0.015 sec
73 20:45:41 select * from v_sales_summary LIMIT 0, 1000 24 row(s) returned 0.016 sec / 0.000 sec

```

INSIGHTS

1. Overall Sales Performance

- The dataset contained **500 orders** across **20 products** and **10 cities** between **2023 and 2024**.
 - The **total revenue** generated during this period exceeded **₹200,000+**, indicating consistent sales activity.
 - On average, the **order value** ranged between **₹50 and ₹1,000**, showing a diverse product price mix.
-

2. Monthly Revenue and Order Volume

- Both **revenue** and **order volume** showed steady growth across months.
 - **Q2 (April–June)** of both years contributed the highest share of total sales.
 - The **months of April and July** recorded the **highest total revenue**, suggesting seasonal demand or successful promotions.
 - **January and February** generally showed lower order volumes, reflecting a post-holiday sales dip.
-

3. Year-wise Analysis

- **2024 outperformed 2023** in total revenue by nearly **20–25%**, indicating business growth.
 - The increase was driven by higher order counts and larger average order values in 2024.
-

4. Product Performance

- **Laptops, Smartphones, and Routers** generated the **highest total revenue** among all products.

- **Accessories** such as **Chargers, Headphones, and Power Banks** had a high frequency of orders but lower revenue contribution — indicating they are low-value, high-volume items.
 - **Projectors and Cameras** were niche items with lower order counts but higher average transaction values.
-

5. City-wise Insights

- **Bangalore, Mumbai, and Kolkata** were the **top-performing cities**, together contributing more than half of total sales.
 - **Jaipur and Lucknow** recorded moderate sales activity.
 - This shows concentration of customer base in major metro cities.
-

6. Customer Insights

- Several customers placed **multiple orders across months**, indicating **repeat purchase behavior**.
 - A few high-value customers contributed disproportionately to revenue, reflecting a **Pareto pattern (80/20 rule)**.
-

7. Business Interpretation

- **Seasonal spikes** in Q2 and Q3 suggest that marketing campaigns or festive offers could be strategically timed.
- Focus can be given to **metro cities** and **top-performing products** for better ROI.
- There is scope to **increase sales in low-performing months** by targeting discounts or promotions.